Does the enforcement of legal play through logit masking and environment constraints accelerate agent convergence?

by

Francois Qian
URN: 6702759/7

A dissertation submitted in partial fulfilment of the
requirements for the award of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

May 2026

Department of Computer Science
University of Surrey
Guildford GU2 7XH

Supervised by: Nishanth Sastry

I declare that this dissertation is my own work and that the work of others is acknowledged and indicated by explicit references.

Francois Qian
May 2026

© Copyright Francois Qian, May 2026

Abstract

1. The convergence of self-play in a pseudo-legal environment toward strict legal play.
2. The impact of logit masking on learning dynamics, specifically regarding the "grokking" of physical board mechanics.

In the development of neural agents for combinatorial games, a fundamental tension exists between explicit policy pruning and emergent rule internalization. This thesis investigates two primary paradigms: **Logit Masking** and **Punishment Propagation**.

- **Hypothesis:** While logit masking provides superior convergence speed, unmasked training (Punishment Propagation) forces the network to "grok" the underlying physical mechanics of the environment, potentially yielding more robust latent representations.

Acknowledgements

I would like to thank my supervisor, Nishanth Sastry, for overseeing the project.

Contents

1 Introduction **14**
1.1 Project Background 14
1.2 Project Description 14
1.3 Aims and Objectives 15
1.4 Thesis Structure 15

2 Literature Review **16**
2.1 Chess as a MDP 16
2.2 Chess played by Machines 16
2.2.1 AlphaZero, MuZero 16
2.2.2 ChessFormer 17
2.3 Invalid Action Masking 17
2.4 Curriculum learning 17
2.5 Learning Dynamics and Grokking 18
2.6 Summary and Research Gap 18

3 System Design and Methodology **19**
3.1 Chess Library 19
3.1.1 Move Generation 19
3.2 Model Architecture 19
3.2.1 Inputs and Outputs 19
3.2.2 Two-Pass Execution Flow 19
3.3 Monte Carlo Tree Search 20
3.4 Self-Play Pipeline 20

4 Implementation Details **21**
4.1 The Chess Client 21
4.1.1 ChessSquare 21
4.1.2 Bitboard 21
4.1.3 ChessBoard 21
4.1.4 ChessPosition 21
4.1.5 ChessGame 21
4.2 The Model 22
4.2.1 Transformer 22
4.2.2 Data 22

4.3 The Monte Carlo Tree Search ... 22
4.3.1 Nodes ... 22
4.3.2 Edges ... 22
4.3.3 Arenas ... 22
4.3.4 The Orchestrator ... 22
4.4 Main() ... 22

5 Results and Evaluation ... 23
5.1 Experimental Setup ... 23
5.2 Ruleset Convergence Analysis ... 23
5.3 Masking and Grokking Analysis ... 23

6 Conclusion ... 24
6.1 Project Evaluation ... 24
6.2 Limitations and Future Work ... 24

7 Ethics ... 25
7.1 Using Typst ... 25
7.1.1 Adding figures ... 25
7.1.2 Adding tables ... 25
7.1.3 Adding equations ... 25

Bibliography ... 27

List of Figures

Figure 1 An example figure, with two parts 26

List of Tables

Table 1 An example table, showing speed in operations per cycle per multiprocessor **25**

Glossary

$Q(s, a)$	The total reward accumulated from state
$C(s)$	CPUCT
S_i	Upper confidence bound
$\overline{x_i}$	The Mean Value of the current node
c	The Exploitation parameter
N	The current node's Visit Count
n_i	The Visit Count of the child

Abbreviations

MCTS	Monte Carlo Tree Search
UCB1	Upper Confidence Bound 1
PUCT	Predictor + Upper Confidence Bound applied to Trees
CPUCT	Exploration Predictor Constant

Chapter 1

Introduction

1.1 Project Background

Modern machine learning often relies on scaling data and compute over opaque architectures. However, these models are often blind to the "rules" of the environment, aided by **Logit Masking** - a heuristic that artificially zeroes out the probability of illegal actions prior to softmax activation. While efficient, this deprives the network of feedback for invalid predictions, bypassing the models need to internally represent the physical constraints of the environment. Furthermore, standard RL agents are protected against making bad decisions through the rules of the environment. This prevents observation of how internal action space pruning to match a more constrained environment before developing strategy influences learning. Chess provides an optimal substrate for this investigation: it presents highly ambiguous intermediate states bound by objective ground truths and a mathematically perfect, yet computationally intractable, solution space.

1.2 Project Description

Existing literature rarely assesses the impact of these environmental guardrails on the learning dynamics of transformer-based agents. This project deconstructs these dynamics by investigating two orthogonal axes of environmental constraint: Action Space Constraints and Logit Masking.

To facilitate explicit measurement of model interpretability, a custom two-pass autoregressive transformer encoder and a bipartite Monte Carlo Tree Search (MCTS) are introduced. This architecture decouples piece selection from destination selection, providing a granular window into the model's spatial reasoning.

The experimental configurations are defined across two axes:

1. **Axis 1: Rule Set Convergence (Action Space Constraints)**
   - **Control:** Training on a strict legal ruleset.
   - **Test:** Training on a pseudo-legal ruleset where explicit king capture is permitted and terminates the game.
   - **Hypothesis:** The network is first bootstrapped learning the rules and goal of the environment. Optimal play in a pseudo-legal environment will naturally converge

to legal play. Concepts such as pins and checkmates will manifest not as hardcoded rules, but as emergent survival heuristics to secure or prevent king capture.

2. **Axis 2: Logit Masking**
   - **Control:** Masked logits (illegal moves are mathematically eliminated prior to selection).
   - **Test:** Unmasked logits (illegal moves are permitted by the network but penalized by the environment/loss function).
   - **Hypothesis:** Unmasked training will incur an initial sample-efficiency penalty but will force the network to internally map the physical mechanics of the board, yielding more robust and generalizable long-term representations.

1.3 Aims and Objectives

The primary aim of this project is to train multiple transformer-based agents via self-play reinforcement learning across varying rule sets and action spaces, quantitatively evaluating their performance, learning trajectories, and emergent behaviors.

Specific objectives include:

- Develop a bespoke, high-performance Rust chess engine supporting both legal and pseudo-legal state generation and validation.
- Architect and implement a two-pass autoregressive transformer and bipartite MCTS to explicitly model piece-then-square action selection.
- Train four distinct model configurations via self-play: (Legal, Masked), (Legal, Unmasked), (Pseudo-Legal, Masked), and (Pseudo-Legal, Unmasked).
- Quantitatively evaluate the convergence rates, illegal move proposal frequencies, and the emergence of rule-abiding play across the unconstrained configurations.

1.4 Thesis Structure

The remainder of this dissertation is structured as follows:

- **Chapter 2** reviews the literature surrounding reinforcement learning, transformer architectures, and action masking.
- **Chapter 3** details the system design, including the bipartite MCTS and two-pass encoder architecture.
- **Chapter 4** outlines the implementation details of the Rust engine and training pipeline.
- **Chapter 5** presents the results and evaluation of the four experimental configurations.
- **Chapter 6** concludes the project and discusses avenues for future work.

Chapter 2

Literature Review

2.1 Chess as a MDP

Chess is a deterministic, zero-sum, perfect-information game. It is formally modeled as a Markov Decision Process (MDP) defined by a tuple (S, A, T, R), where S is the state space ($|S| \approx 10^{\{40\}}$), A is the action space, $T : S \times A \to S$ is the deterministic transition function, and $R : S \times A \to \{-1, 0, 1\}$ is the reward function (Shannon, 1950).

In standard reinforcement learning (RL) implementations, the environment strictly enforces a legal action space $A_{L(s)} \subset A$, where A_L contains only moves that comply with the rules of chess (e.g., preventing self-check) (World Chess Federation (FIDE), 2023). This project introduces a relaxed pseudo-legal action space $A_{P(s)}$, defined as the set of all geometrically valid piece movements regardless of king safety. Under A_P, the termination condition is shifted from the abstract concept of "checkmate" to the explicit physical capture of the King (K_{cap}).

For any state s, an optimal policy π^* trained in A_P must implicitly learn to prune the action space, such that:

$$\pi^*(a \mid s) \to 0 \quad \text{for all } a \in \left(A_{P(s)} \setminus A_{L(s)}\right)$$

In this paradigm, concepts such as pins and checkmates are not hardcoded environmental constraints, but emergent survival heuristics within the value landscape.

2.2 Chess played by Machines

2.2.1 AlphaZero, MuZero

The paradigm of self-play RL in chess was defined by AlphaZero (Silver *et al.*, 2017), which combined Monte Carlo Tree Search (MCTS) with deep convolutional neural networks (CNNs) for policy and value evaluation. However, AlphaZero relies entirely on a hardcoded environment to generate $A_{L(s)}$, bypassing the need for the network to internalize the rules of the game.

Subsequent architectures, such as MuZero (Schrittwieser *et al.*, 2020), removed the need for a known transition dynamics model by learning a latent environment simulator. While MuZero demonstrates the ability to adhere to environment constraints implicitly

within its hidden state transitions, its policy head still simulates within the legal action space during MCTS rollouts; the literature lacks insight into how the removal of these environmental guardrails impacts the sample efficiency and representational robustness of the underlying network.

2.2.2 ChessFormer

Recent advancements have demonstrated the efficacy of Transformer architectures in chess, shifting away from the spatial inductive biases of CNNs. (Monroe and Chalmers, 2024) demonstrated that Transformers can achieve Grandmaster-level performance without explicit search, relying entirely on attention mechanisms to evaluate board states. Furthermore, (Jenner *et al.*, 2024) provided evidence of learned look-ahead within chess-playing neural networks, showing that attention heads naturally encode future board states.

Standard chess engines flatten the action space into a single discrete distribution (e.g., 4,672 possible moves). To explicitly measure spatial reasoning and rule internalization, this project utilizes a custom two-pass autoregressive Transformer encoder. By decoupling piece selection $P(\text{from} \mid s)$ and destination selection $P(\text{to} \mid s, \text{from})$, the architecture forces the attention mechanism to explicitly map the geometric constraints of the selected piece, providing a granular window into the model's internal representation of board topology.

2.3 Invalid Action Masking

In policy gradient algorithms, it is standard practice to prevent the selection of illegal actions via Invalid Action Masking (Logit Masking). Masking applies a transformation to the policy logits z prior to the softmax activation:

$$P(a_i \mid s) = \frac{\exp(z_i + M_i)}{\sum_j \exp\big(z_j + M_j\big)}$$

where $M_i = -\infty$ if $a_i \notin A_{L(s)}$, and 0 otherwise.

While usually done on intuition, and to ease implementation, (Huang and Ontañón, 2020) demonstrated that invalid action masking significantly improves sample efficiency and asymptotic performance in environments with large, state-dependent action spaces. (Kanervisto, Scheller and Hautamäki, 2020) similarly emphasised the importance of action space transformations to create learnable environments, especially for continuous action spaces. However, masking renders the network "blind" to the rules; the invalidity of an action is treated as an external mathematical absolute rather than a learned feature.

2.4 Curriculum learning

(Bengio *et al.*, 2009) showed the importance of incremental learning… can i say something here? i want to say training on pseudo legal and unmasked is incremental learning like, learn how pieces move, then learn to capture the king, then learn to checkmate and keep own king safe. my mcts implementation seems to shortcut this in a detrimental way however…

2.5 Learning Dynamics and Grokking

The requirement for an unmasked network to learn the rules of chess from scratch introduces complex learning dynamics. (Power *et al.*, 2022) identified the phenomenon of "grokking," where neural networks trained on small algorithmic datasets exhibit delayed generalization long after overfitting the training data. This was possible because of weight decay pressure, illustrating the importance of weight regularisation.

2.6 Summary and Research Gap

While the efficacy of action masking is well-documented in general RL (Huang and Ontañón, 2020), and Transformers have proven capable of modeling chess (Jenner *et al.*, 2024), the intersection of these domains remains unexplored. Existing literature does not isolate the impact of environmental guardrails on the representational quality of Transformer-based MCTS agents.

In the context of unmasked, pseudo-legal chess training, the network faces a dual-optimization problem: it must learn the physical constraints of the board (the rules) and the strategic evaluation of states (the game). It is hypothesized that the network will exhibit a grokking-like phase transition: an initial period of high illegal-move proposal (random walk), followed by a sharp convergence toward $A_{L(s)}$ as the physical constraints are internalized, preceding the development of high-level strategy.

This project addresses this gap by training four distinct configurations across two orthogonal axes: Action Space Constraints (Legal vs. Pseudo-Legal) and Logit Masking (Masked vs. Unmasked). By utilizing a bespoke two-pass autoregressive encoder, this research provides a novel, quantitative analysis of how rule internalization and action space pruning manifest within the attention layers of a chess-playing agent.

Chapter 3

System Design and Methodology

The code for this project contains 3 main components. SUB

3.1 Chess Library

A bespoke client was developed in Rust to meet the high-performance demands of MCTS rollouts and ensure memory safety during concurrent self-play.

3.1.1 Move Generation

Ray casting is optimized using constant directional masks and pre-computed "between" masks, allowing for $O(1)$ validation of sliding piece moves.

3.2 Model Architecture

A 2 pass autoregressive encoder was chosen as a means to expose model interpretability in a measurable way. The same policy head is re used to generate both pieces to select from in the first pass, then squares to move to given an extra input plane in the first pass one hot encoded with the selected piece. The model is given a 64×14 tensor representing the 12 chessboard bitboards, the en passant square, and the optional selected square. [cls] token

3.2.1 Inputs and Outputs

- **Inputs:** 12 piece bitboards, 1 en passant bitboard, 1 selected square bitboard, castling rights (4x1 f32), 50-move rule scalar (1 f32).
- **Outputs:** Policy head (64-square distribution) and Value head (Win/Draw/Loss buckets). Both utilize softmax activation and KL divergence loss.

3.2.2 Two-Pass Execution Flow

1. **Pass 1:** Select origin square.
2. **Pass 2:** Select destination square (Pass 1 output fed back into input). Promotions are handled automatically during MCTS rollouts.

3.3 Monte Carlo Tree Search

A custom bipartite mcts search algorithm is architected to support the 2 pass encoder. The MCTS utilizes the PUCT (Predictor + Upper Confidence Bound applied to Trees) algorithm to balance exploration and exploitation. Dirichlet noise is added at the root node to encourage exploration of other lines.

$$\alpha(\iota, x_2) = \sum_{x_1, d_{\iota-1}} \alpha(\iota - 1, x_1) \Pr\left\{ \boldsymbol{r}_{n(\iota-1)+x_1, n\iota+x_2}, \boldsymbol{t}_{\iota-1} \right\}$$

3.4 Self-Play Pipeline

Multiple games are vectorised and initialised with their own monte carlo tree search. Each simulation occurs independently of each other and in parallel via Rayon. The network is updated via backpropagation using autograd, minimizing the combined policy and value loss.

Chapter 4

Implementation Details

4.1 The Chess Client

4.1.1 ChessSquare

4.1.2 Bitboard

The board is represented using 12 primary bitboards (unsigned 64-bit integers), packing the state into 96 bytes. Additional convenience bitboards (occupancy masks) are maintained to accelerate move generation. Bitboards enable high-performance move generation via bitwise intrinsics. In deep learning, these are expanded into 64×14 tensors.

4.1.3 ChessBoard

4.1.4 ChessPosition

4.1.5 ChessGame

```
pub struct ChessGame {
    pub castling_rights: CastlingRights,
    pub chessboard: ChessBoard,
    pub en_passant: Option<ChessSquare>,
    pub fullmove_counter: u32,
    pub game_history: alloc::vec::Vec<GameStateEntry>,
    pub halfmove_clock: u32,
    pub outcome: Outcome,
    pub rule_set: RuleSet,
    pub side_to_move: chess_piece::Color,
    pub zobrist_hash: u64,
}
```

4.2 The Model

4.2.1 Transformer

4.2.2 Data

4.3 The Monte Carlo Tree Search

4.3.1 Nodes

4.3.1.1 Select

4.3.1.2 Move

4.3.2 Edges

4.3.3 Arenas

4.3.4 The Orchestrator

4.4 Main()

Chapter 5

Results and Evaluation

5.1 Experimental Setup

To ensure fairness, all variants were initialized with identical seeds, trained for the same number of epochs, and evaluated at identical MCTS search depths.

5.2 Ruleset Convergence Analysis

5.3 Masking and Grokking Analysis

Chapter 6

Conclusion

6.1 Project Evaluation

6.2 Limitations and Future Work

- **Compute Constraints:** Hardware limitations restricted the total number of training epochs, preventing observation of

deep late-stage grokking.

- **Move Generation:** Future iterations should replace standard bitboard raycasting with Magic Bitboards for optimal

performance.

- **Endgame Tablebases (EGTB):** Injecting perfect knowledge at varying stages of the training pipeline remains a highly

viable area for future research to bootstrap the value head and sharpen endgame play.

Chapter 7

Ethics

7.1 Using Typst

Operation	Speed
Add, Mul, Mul-Add	8
Reciprocal	2
Divide	0.88
Divide Intrinsic	1.6
Recip. Square Root	2
Square Root	1
Logarithm	2
Exponent	1
Sin, Cos Intrinsics	2
Sin, Cos, Tan	Slow

Table 1: An example table, showing speed in operations per cycle per multiprocessor

7.1.1 Adding figures

Figures are added using the `#figure` function. Typst automatically handles the numbering and placement of your images, as shown in the example that produces Figure 1.1. Common formats like PNG, JPEG, and SVG are supported natively.

7.1.2 Adding tables

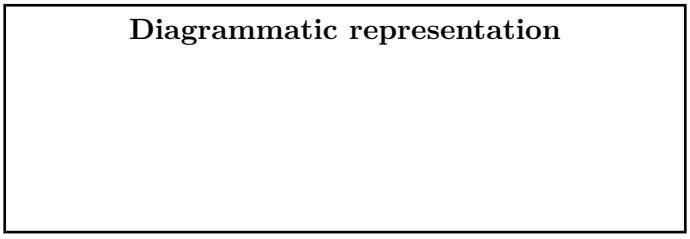
Tables are defined directly in the source code using the `#table` function. This provides a highly flexible way to grid your data. An example of a styled table is given in Table 1.1.

7.1.3 Adding equations

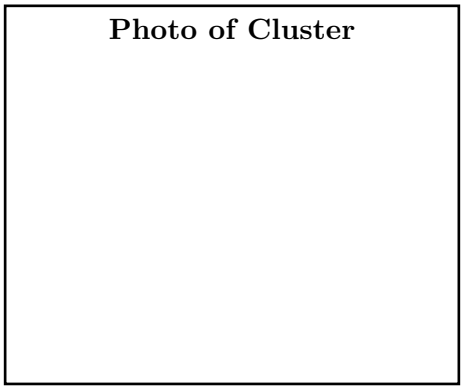
A primary advantage of Typst is its intuitive mathematical notation. Equations can be written within `$ ... $` delimiters. It handles numbered equations easily, as in the recursive formula:

$$\alpha(\iota, x_2) = \sum_{x_1, d_{\iota-1}} \alpha(\iota - 1, x_1) \Pr\left\{\boldsymbol{r}_{n(\iota-1)+x_1, n\iota+x_2}, \boldsymbol{t}_{\iota-1}\right\}$$

Inline math is also supported, for example to specify $1 \leq \iota < N$. Typst's syntax remains readable even for complex multiline expressions: testing testings



(a) Diagrammatic representation



(b) Photo of the Tempest cluster

Figure 1: An example figure, with two parts

$$\Pr\left\{\boldsymbol{r}_{0,n\iota+x_2}, \sigma_{n\iota} = x_2\right\} = \sum_{x_1, d_{\iota-1}} \left[\Pr\left\{\boldsymbol{r}_{0,n(\iota-1)+x_1}, \sigma_{n(\iota-1)} = x_1\right\} \times \Pr\left\{\boldsymbol{r}_{n(\iota-1)+x_1,n\iota+x_2}, \boldsymbol{t}_{\iota-1}\right\}\right]$$

Bibliography

Bengio, Y. *et al.* (2009) "Curriculum learning," in *Proceedings of the 26th Annual International Conference on Machine Learning*. Montreal, Quebec, Canada: Association for Computing Machinery (ICML '09), pp. 41–48. Available at: https://doi.org/10.1145/1553374.1553380.

Huang, S. and Ontañón, S. (2020) "A Closer Look at Invalid Action Masking in Policy Gradient Algorithms," *CoRR* [Preprint]. Available at: https://arxiv.org/abs/2006.14171.

Jenner, E. *et al.* (2024) *Evidence of Learned Look-Ahead in a Chess-Playing Neural Network*. Available at: https://arxiv.org/abs/2406.00877.

Kanervisto, A., Scheller, C. and Hautamäki, V. (2020) "Action Space Shaping in Deep Reinforcement Learning," in *2020 IEEE Conference on Games (CoG)*, pp. 479–486. Available at: https://doi.org/10.1109/CoG47356.2020.9231687.

Monroe, D. and Chalmers, P.A. (2024) *Mastering Chess with a Transformer Model*. Available at: https://arxiv.org/abs/2409.12272.

Power, A. *et al.* (2022) "Grokking: Generalization beyond overfitting on small algorithmic datasets," *arXiv preprint arXiv:2201.02177* [Preprint].

Schrittwieser, J. *et al.* (2020) "Mastering Atari, Go, chess and shogi by planning with a learned model," *Nature*, 588(7839), pp. 604–609. Available at: https://doi.org/10.1038/s41586-020-03051-4.

Shannon, C.E. (1950) "XXII. Programming a computer for playing chess," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314), pp. 256–275. Available at: https://doi.org/10.1080/14786445008521796.

Silver, D. *et al.* (2017) *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. Available at: https://arxiv.org/abs/1712.01815.

World Chess Federation (FIDE) (2023) "FIDE Laws of Chess". Available at: https://handbook.fide.com/chapter/E012023.